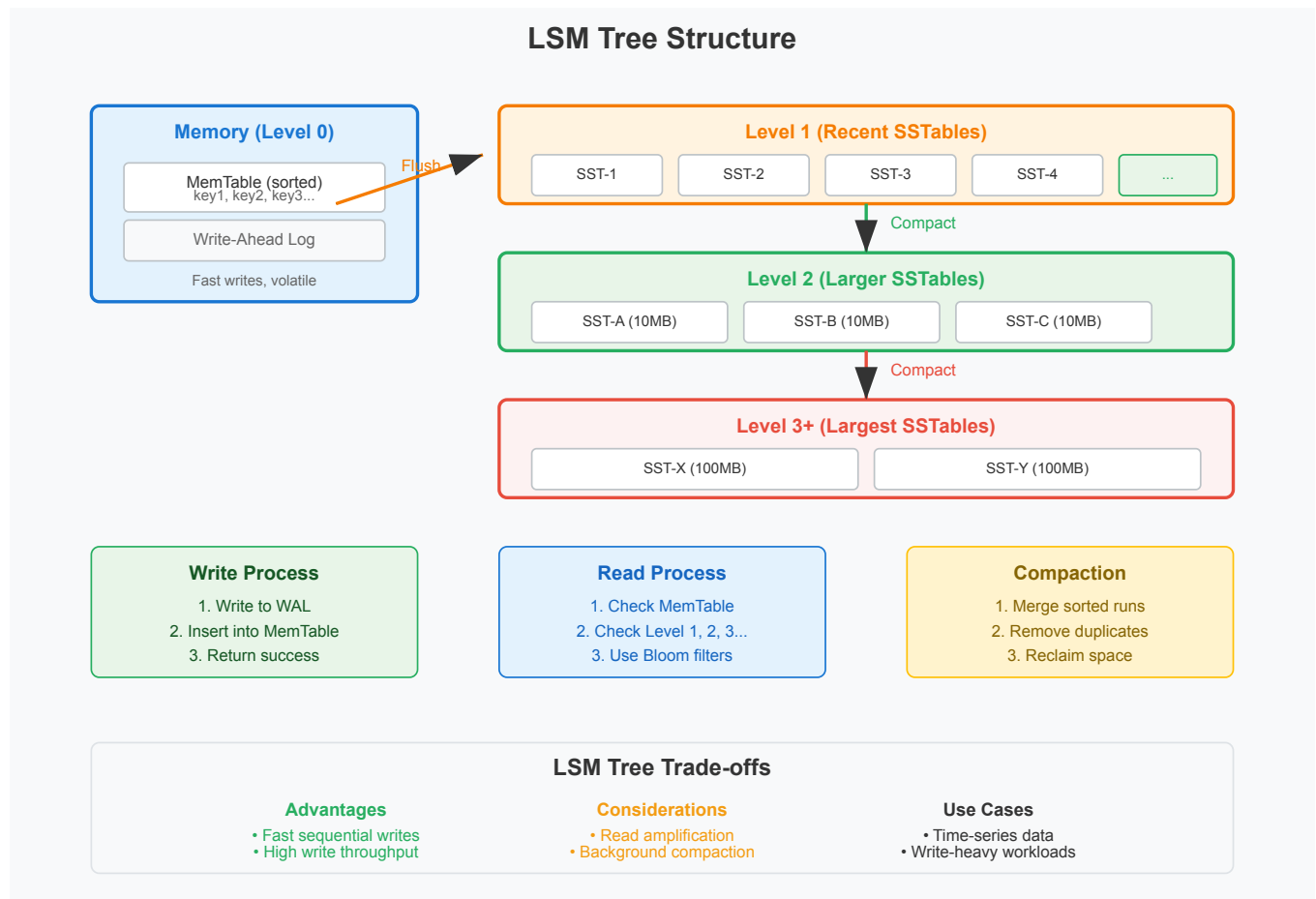


LSM Trees | System Design Primer

systemdesignschool.io/fundamentals/lsm-tree



LSM Tree

[SSTables](#) provide sorted, immutable storage files but create a new problem: how do you efficiently handle continuous writes without creating a lot of small files? Log-structured merge (LSM) trees solve this by organizing SSTables into a hierarchy that balances write throughput with read performance.

Traditional B-tree engines suffer from write amplification because each small change requires reading pages, modifying them, and writing them back to random disk locations. LSM trees eliminate this bottleneck by batching writes in memory and converting them to efficient sequential disk operations using SSTables as the storage format.

Core Architecture

Memory-first design

The core insight is deferring disk work until it's efficient. Instead of immediately updating disk structures, LSM trees buffer writes in a memory table (memtable) and periodically flush sorted batches to disk as immutable [SSTable](#) files. Background compaction merges overlapping files to maintain read performance. This architecture transforms random write patterns into predictable sequential operations that modern storage handles efficiently.

Why batching works

Sequential writes dramatically outperform random access. Creating a new SSTable for every write would be catastrophic - **adding one key-value pair to an existing 1GB SSTable requires reading the entire file, merging the new entry, and writing over 1GB back to disk**. Patrick O'Neil's 1996 paper introduced the solution: **buffer writes in memory, flush them as sorted runs, and merge those runs when the system has spare I/O capacity**.

Leveled Organization

File hierarchy

Without structure, frequent flushing creates thousands of tiny files that destroy read performance. LSM trees use a pyramid hierarchy where fresh data starts at Level 0 and older data migrates to deeper levels through background compaction.

Consider this progression:

- Memtable fills up → flushes to create a new Level 0 file (newest data)
- After several flushes → Level 0 accumulates 4-8 separate files
- Background compaction → merges Level 0 files into different Level 1 files
- Level 1 grows large → compacts into separate Level 2 files (typically when 10x larger)
- Pattern continues → each level contains different files, with Level 0 always holding the newest data

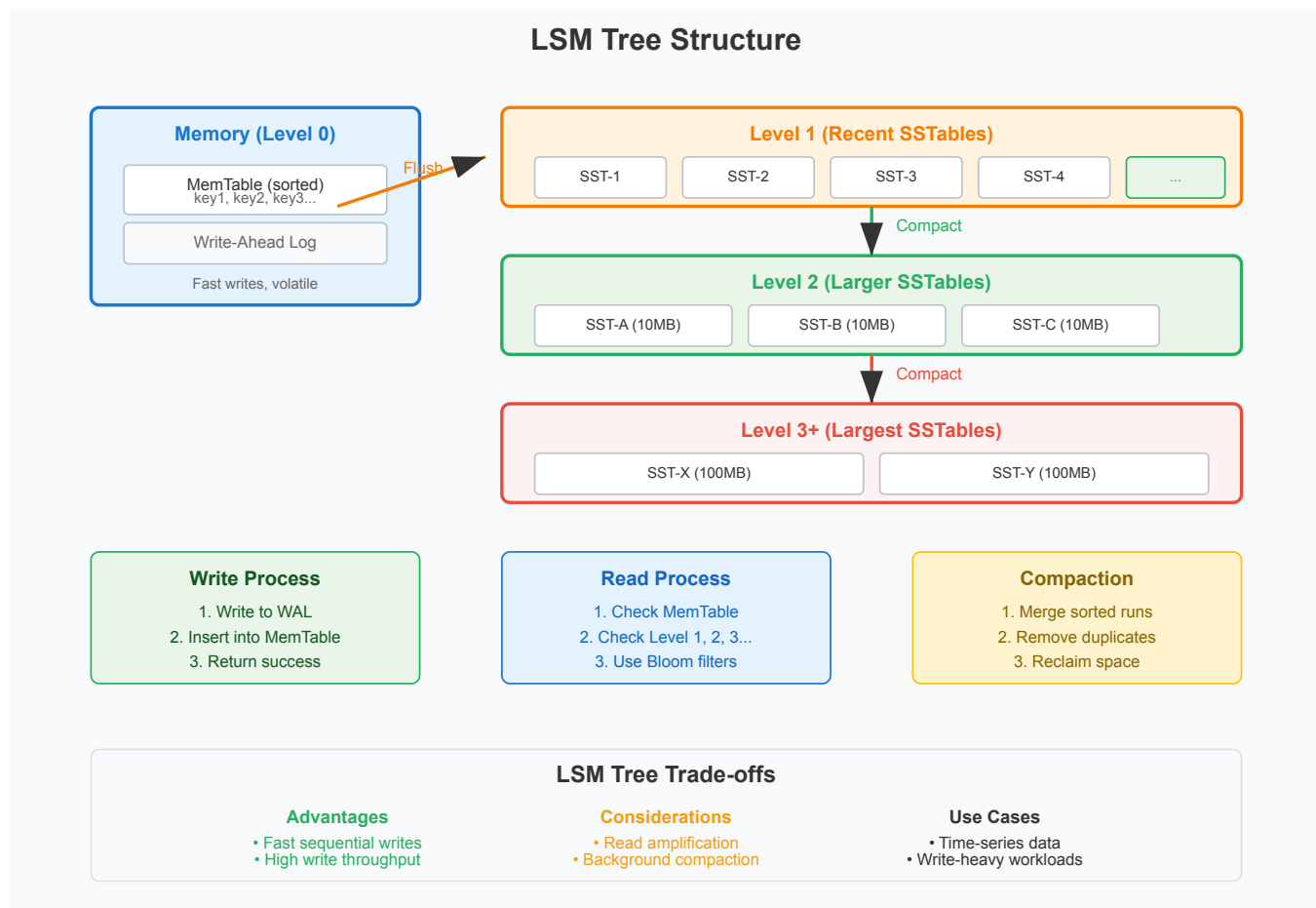
Bounded lookup cost

This hierarchy solves the file proliferation problem. **Without levels, a single lookup might check hundreds of files. With levels, reads check at most 10 files in Level 0, plus one file per deeper level**. The bounded lookup cost maintains query latency even as the database grows to terabytes.

Production adoption

Bigtable popularized the pattern in production, and today Cassandra, DynamoDB, and RocksDB rely on LSM variants for sustained write throughput. The trade-off is increased complexity - reads must check multiple locations and background compaction consumes significant I/O bandwidth.

An LSM tree consists of a memtable for recent writes, immutable SSTables on disk, and a write-ahead log for crash recovery. The write-ahead log (WAL) records each operation before acknowledging it, ensuring durability while the memtable optimizes read-after-write latency.

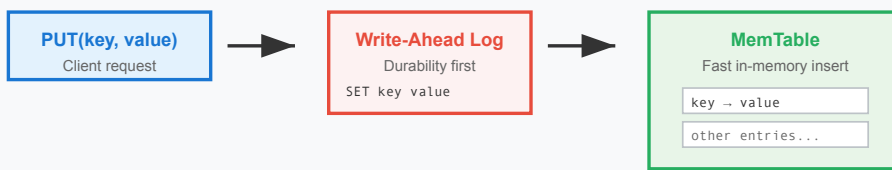


Write operations

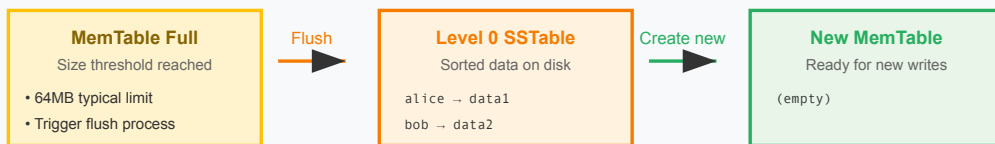
Writes prioritize immediate acknowledgment over disk efficiency. Each write operation appends to the WAL, updates the memtable, and returns success immediately. This approach minimizes write latency while preserving durability. When the memtable reaches a configured size threshold (typically 64MB to 256MB), the engine flushes its contents to disk as a new Level 0 SSTable. Since SSTables are immutable, the flush process requires no coordination with concurrent readers.

LSM Tree Write Flow

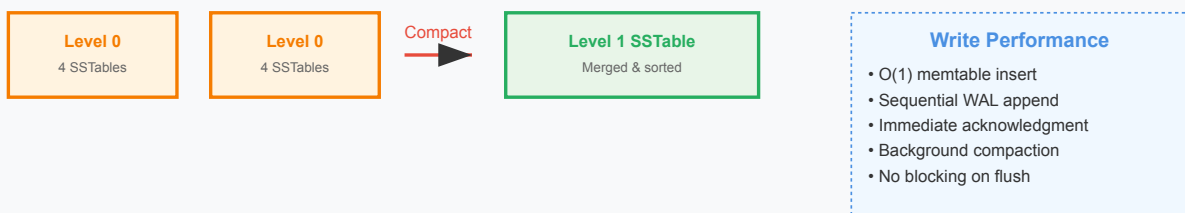
1. Write Request



When MemTable Gets Full



Background Compaction



Write-Ahead Log (WAL)

The write-ahead log solves the durability problem inherent in memory-based structures. Memtables disappear during crashes, potentially losing all writes since the last flush. The WAL prevents this by recording every operation in an append-only disk file before acknowledging the write.

For example, a WAL entry may look like this:

- 2024-01-15 14:30:25.123 PUT user:123 "alice"
- 2024-01-15 14:30:25.456 DELETE user:456
- 2024-01-15 14:30:25.789 PUT session:abc "active"

Recovery simply replays all WAL entries since the last successful flush, reconstructing the memtable state. WAL entries become redundant once their corresponding data reaches persistent storage as SSTables, allowing periodic truncation.

Updates and deletes

LSM trees maintain immutability by treating updates and deletes as new entries rather than modifications. An update writes a newer value for the same key. A delete writes a tombstone marker indicating the key should be considered removed. During reads, the engine searches from newest to oldest data, so recent entries automatically shadow older ones.

Tombstones and Read Correctness

Write Sequence



Benefits

- No in-place edits
- Readers see consistent snapshots
- Tombstones eventually removed
- Space reclaimed during compaction
- Newer values shadow older ones

Storage Layout (Newest → Oldest)



Read Process
Tombstone found! Check newest first
Return: key deleted



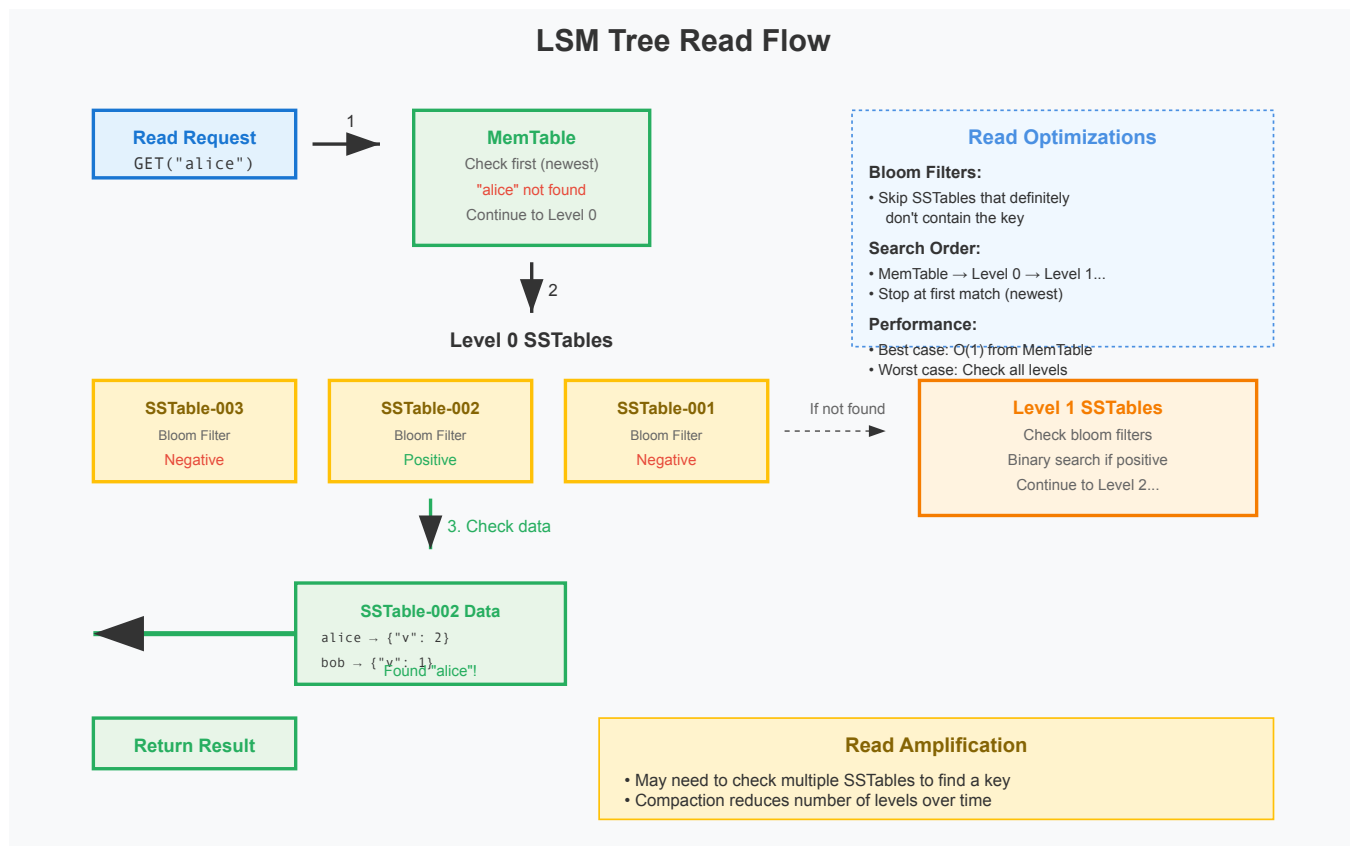
Immutable SSTables use logical deletion: newer records shadow older ones without modification



This approach preserves the immutable nature of SSTables while supporting the full spectrum of database operations. The trade-off is that obsolete data continues consuming disk space until background compaction removes it.

Read operations

Reads must check multiple data locations. The read path starts with the memtable for the most recent writes, then searches SSTables from newest to oldest until finding the key or exhausting all possibilities. [Bloom filters](#) eliminate most unnecessary disk access by quickly determining which SSTables definitely don't contain a key. Sparse indexes and block caches further optimize the lookup process.



Range scans exploit the sorted layout by streaming across files once compaction has merged overlapping ranges. The read path complexity increases with the number of levels and files, but bounded file counts per level keep worst-case latency predictable.

Compaction

Background compaction prevents file proliferation and maintains read performance. Without maintenance, writes create hundreds of small SSTable files that destroy lookup performance. Compaction periodically merges overlapping files into larger, consolidated SSTables. The process follows a predictable pattern: identify overlapping files, stream them in sorted key order, and emit a new SSTable containing the most recent version of each key.

When multiple files contain the same key, sequence numbers determine precedence. Tombstones override older live values. The merge creates a new SSTable with fresh metadata, bloom filters, and checksums. Only after the new file reaches disk does the storage engine update its manifest to replace the input files.

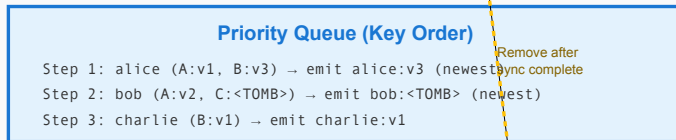
LSM Compaction Flow

1. Select Overlapping Runs



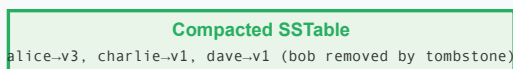
Merge

2. Priority Queue Merge



Write

3. New Merged SStable



4. Cleanup

Delete input files

Consistency Guarantees

During Compaction:

- Input SStables remain untouched
- Readers see consistent catalog
- New file built completely first

Merge Rules:

- Newest value wins for same key
- Tombstones remove older versions
- Output maintains sorted order

After Completion:

- Manifest updated atomically
- Input files marked for deletion
- Readers switch to new file

Benefits:

- Fewer files to check on reads
- Space reclaimed from deletes
- Improved sequential locality



Write amplification trade-offs

Write amplification is the fundamental trade-off. **Each byte gets rewritten multiple times as it moves through the level hierarchy.** RocksDB typically sees 10-15x write amplification on modern NVMe storage. This cost is amortized across many operations and happens in background threads without blocking user requests.

Compaction strategies

Compaction strategies depend on workload patterns. Leveled compaction maintains one sorted run per level, minimizing read amplification but increasing write amplification. Tiered compaction allows multiple runs per level, reducing write costs but requiring more file checks during reads. Production systems tune level sizes, fanout ratios, and compaction concurrency to balance these trade-offs.

Production considerations

Operational challenges

Heavy write bursts, node repairs, or failed compactions create many overlapping Level 0 files. Reads continue working but traverse more bloom filters and disk blocks, increasing tail latency. Operators respond by throttling writes or adding compute resources to let compaction catch up. Monitoring Level 0 file counts and pending compaction work provides early warning before performance degrades.

A single partition key receiving most writes can saturate one shard's WAL and memtable capacity. Production clusters distribute hot keys across multiple partitions or implement adaptive load shedding to prevent compaction debt accumulation.

Tuning and monitoring

Level size ratios, target file sizes, and compaction strategies significantly impact performance. Files that are too small trigger constant rewrites. Files that are too large slow recovery and increase read latency. Write amplification monitoring, combined with read latency percentiles, guides operational decisions.

When to use LSM trees

Time-series ingestion, logging pipelines, and message queues benefit from sustained write throughput and predictable latency. Read-heavy workloads with strong locality often perform better with B-trees, which avoid multi-level lookups and compaction overhead. The choice depends on whether your bottleneck is write throughput or read efficiency.

LSM Tree Implementation with Compaction

```
from collections import OrderedDict
from typing import Dict, List, Optional, Tuple

def __init__(self, entries: List[Tuple[str, Optional[str]]], level: int):
    self.entries: Dict[str, Optional[str]] = dict(entries)
    self.level = level

def get(self, key: str) -> Optional[str]:
    return self.entries.get(key)

def __init__(self, memtable_limit: int = 4, fanout: int = 2):
    self.memtable: "OrderedDict[str, Optional[str]]" = OrderedDict()
    self.wal: List[Tuple[str, Optional[str]]] = []
    self.levels: List[List[Run]] = []
    self.memtable_limit = memtable_limit
    self.fanout = fanout

def put(self, key: str, value: str):
    self._write(key, value)

def delete(self, key: str):
    self._write(key, None)

def _write(self, key: str, value: Optional[str]):
    self.wal.append((key, value))
    self.memtable[key] = value
    print(f"memtable[{key}] = {value}")
    if len(self.memtable) >= self.memtable_limit:
        self._flush()

def get(self, key: str) -> Optional[str]:
    # Check memtable first
```

```
if key in self.memtable:  
    return self.memtable[key]
```

The implementation demonstrates the core mechanics: writes batch in memory, flushes create sorted runs, compaction merges overlapping data, and tombstones handle deletes without mutating existing SSTables.